

TEKNOFEST

HAVACILIK, UZAY VE TEKNOLOJİ FESTİVALİ

ROBOTAKSİ – BİNEK OTONOM ARAÇ YARIŞMASI

KRİTİK TASARIM RAPORU

(HAZIR ARAÇ KATEGORİSİ)

TAKIM ADI: Göbeklitepe Otonom Sürüş Takımı

TAKIM ID: 465491

İÇERİK

1. Takım Organizasyonu	3
2. Ön Tasarım Raporu Değerlendirmesi	4
3. Hazır Araç Özellikleri ve Analizi	5
4. Araç Dinamiği Modelleme/Tanımlama ve Kontrolü	6
5. Araç Kontrol Ünitesi	8
6. Otonom Sürüş Algoritmaları	9-13
7. Güvenlik Önlemleri	13-15
8. Simülasyon	15-17
9. Sistem Entegrasyonu	17-19
10. Test ve Doğrulama	20-22
11. Referanslar	22-24



1. Takım Organizasyonu

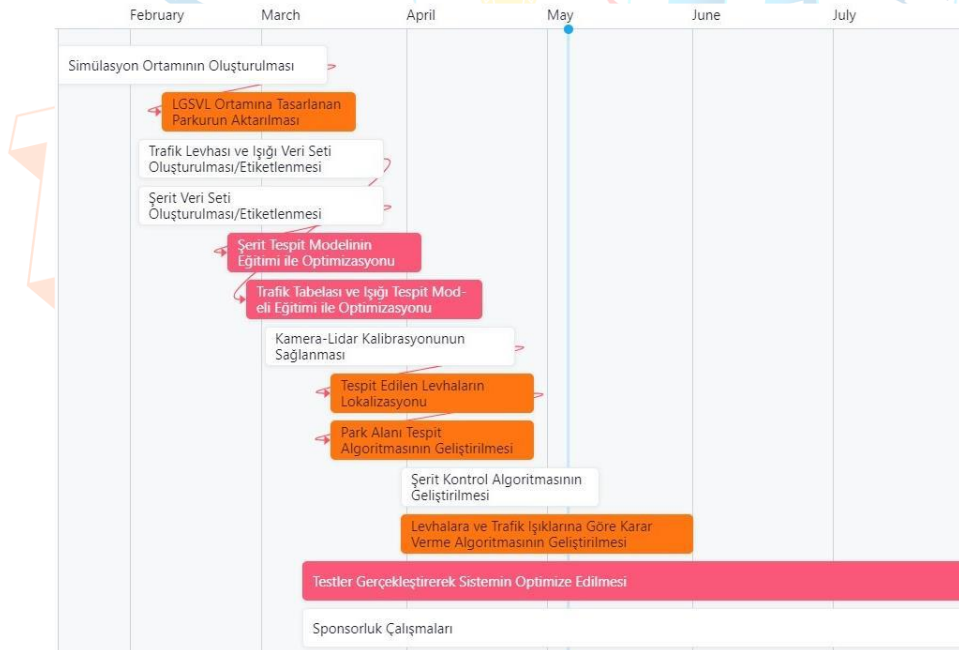
Harran Üniversitesi bilgisayar mühendisliği bölümü ağırlıklı olmak üzere farklı üniversite, bölüm ve lise öğrencilerinden oluşan multidisipliner yapıya sahip bir takımız. Ekip, otonom sürüş algoritmalarını, yapay zeka modellerini, simülasyon teknolojilerini takip eden, bu alanda farklı proje ve staj programlarında yer alarak kendilerini otonom araçlar konusunda geliştirmekte olan 7 kişilik bir öğrenci grubundan oluşmaktadır.

Projemiz kapsamında teknik geliştirmeler için 3 temel alt grup oluşturulmuştur. Alt gruplar sayesinde hem iş paketlerinin takibi hem de görev dağılımının kontrol edilmesi kolaylaştırılmıştır. Bu alt gruplar şekil-1.1 de görüldüğü üzere ROS takımı (kontrol algoritmaları), Görüntü İşleme takımı (Tabela tanıma, şerit tanıma, lidar ile nesne algılama) ve Simülasyon Takımından oluşmaktadır.

Simülasyon ekibi, simülasyon ortamı için gerekli olan harita tasarımını gerçekleştirip, Teknofest tarafından verilecek olan aracın bu haritaya entegre edilmesi, araçta bulunması gereken sensörlerin konumlandırılması ve oluşturulan algoritmaların simülasyon ortamında testlerinin yapılmasından sorumludur.

ROS ekibi, aracın kontrol algoritmasından, canbus haberleşmesinden ve araç bilgisayarı entegrasyonundan sorumludur. Otonom kontrol algoritması yardımıyla park görevi ve kontrol geliştirmeleri gerçekleştirilmektedir.

Görüntü İşleme ekibi, parkur şerit çizgilerinin tespiti, trafik ışık ve levhalarının tespiti ve lokalizasyonundan sorumludur. Ayrıca sürüş için gerekli sürülebilir rotanın oluşturulması ve park etmeye yönelik parametreler döndürmekten sorumludur.



Şekil-1.2: İş-zaman çizelgesi.

2. Ön Tasarım Raporu Değerlendirmesi

2.a. Sensör güncellemesi: ÖTR de belirtmiş olduğumuz park sensörü için GPS kullanmak yerine ZED kamerasının içerisinde yer alan IMU özelliklerinden yararlanmayı bu teknikler ile görüntü işleme ve lidar tekniklerini birleştirmeyi bu sayede park görevini gerçekleştirmeyi sağlıyoruz.

2.b. Simülasyon ortamı güncellemesi: Araç ve harita üzerinde revizyonlar yapılarak ihtiyaç duyulan, kullanılması planlanan sensörler ile simülasyon ortamında düzenlemelere gidildi.

2.c. Otonom sürüş algoritmaları güncellemesi: Simülasyonda ara katman olarak kullandığımız ROS (Robot Operating System) için tercih ettiğimiz versiyon noetic sürümünden oluşuyordu. Araç üzerinde Nvidia Xavier AGX kartı kullandığımız için ve Xavier en son ubuntu 18.04 ortamını desteklediği için ROS'un Melodic sürümüne geçiş sağladık. Aynı zamanda Lidar sensörü ile dubaların algılanması yerine kamera ile şerit takibi yapmayı tercih ettik bu doğrultuda algoritmamızı ve modelimizi güçlendirdik. Bununla birlikte aracımız yolda kararlı bir şekilde kalabildi. Ayrıca tabelaların araca olan uzaklığını tespit etmek için ZED kamerasının sağlamış olduğu derinlik özelliğinden yararlanıyoruz. Lidar sensörü güvenlik önlemi olarak çarpma koruma işlemlerinde kullanılmaktadır.

2.d. ÖTR geri bildirim düzenlemesi: Verilen geri bildirimde “İki nokta LİDAR gürültüsü de olabilir.” şeklinde yapılan uyarıyı dikkate aldık. Buna yönelik şerit takip sistemimizde algılanan noktaları gruplandırıp olası hatalara karşı tekrar bir gruplandırma gerçekleştirdik. Bu sayede olası gürültülerin önüne geçildi ve başarılı bir yol tutuşu sağlandı.

3. Hazır Araç Özellikleri ve Analizi

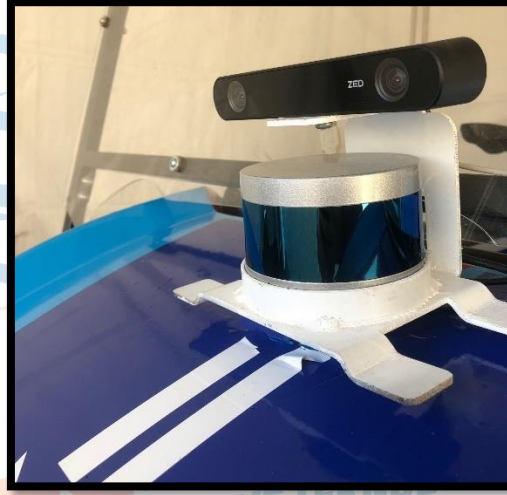
Robotaksi - Hazır araç kategorisi kapsamında üretilmiş olan aracımızın şasisi yüksek mukavemetli uçak sınıfı alüminyum malzeme ile güçlü ve hafif olacak şekilde geliştirilmiştir. Şekil-3.1 de görüldüğü gibi toplam uzunluk ve genişlik avantajları kullanılarak 2 kişinin konforlu bir şekilde yolculuk yapabilmektedir. Arkadan itişli olan aracımız 7,4 kW Schaubmuller motora sahiptir. Olası bir sapma sonucu yaşanacak kazada hasarı minimuma indirmek için canbus protokolü üzerinden gönderilen speed parametresinin minimum ve maksimum değerleri sınırlandırıldı.



TRAGGER T-Car Teknik Veriler		TRAGGER T-Car Ölçüler	T-Car 2
Motor	7,4 KW Schaubmuller	Dingil Mesafesi	2400
Tork /Nm (Max)	125 NM	Toplam Uzunluk	2385
Çalışma Gerilimi / V	48 Volt	Toplam Uzunluk Geril Oturma Aparatıyla Birlikte	2825
Batarya	Trojan T145 Plus	Ön Uzunluk	290
Oturma Kapasitesi	2,4,6,8	Arka Uzunluk	595
Max Araç Hızı Km/h	24	İlave Ters İki Koltuklu Ölçü	1035
Şasi	6000 Serisi Yüksek Mukavemetli Uçak Sınıfı Alüminyum	Şasi Yerden Yüksekliği (Kanopsisiz)	185
Süspansiyon Sistemi		Toplam Yükseklik (Kanopsisiz)	1240
Ön Süspansiyon	Bağımsız / Teleskopik Amortisör	Toplam Yükseklik (Kanopili)	1950
Arka Süspansiyon	Helezon yaylı sabit aks / Teleskopik Amortisör	Yükleme Zemini Yerden Yükseklik	680
Lastikler	10" Duro	Toplam Genişlik (Kanopsisiz)	1330
Fren Sistemi		Arka İz Genişliği	1115
Park Freni	Mekanik / Ayak Pedalı	Ön İz Genişliği	1030
Servis Freni	Hidrolik / Kampana		

Şekil-3.1: Araç genel görünümü ve özellikleri.

Aracın yüksekliği lidar sensörünün ve kameranın konumlandırılması için çok yüksek olduğundan konumlandırma işlemi şekil-3.2 de görüldüğü gibi aracın ön kaporta bölümüne yapılmıştır.



Şekil-3.2: Velodyne Lidar ve ZED 2 Kamera konumlandırılma görseli.

Sensörlerin bu şekilde konumlandırılmasıyla birlikte lidarın gereksiz nokta bulutları toplamasının önüne geçildi ve kameraya eğim verilerek güneşin olumsuz etkileri azaltılmaya çalışıldı.

ZED 2 Kamera geniş açı, derinlik ve IMU özellikleri sayesinde tabela ve şerit algılamasında kolaylık sağlarken, Velodyne lidar (Velodyne Puch Hi-Res 16) çevre faktörlerinin algılanması konusunda 360 derece nokta bulutu algılamasını sağlayabilmektedir. Bu anlamda sürüş alanına yönelik algılama yapılması ekibimiz tarafından tercih edilmektedir.

Otonom sürüş sağlanabilmesi için mekanik bir sistemin elektronik olarak kontrolünü sağlayacak şekilde hazır araçta tarafımıza elektronik direksiyon (by-wire) olarak adlandırılan otonom direksiyon sistemi sağlandı. Bu direksiyon sisteminin özellikleri Tablo-3.1'deki gibidir.

4. Araç Kontrol Ünitesi

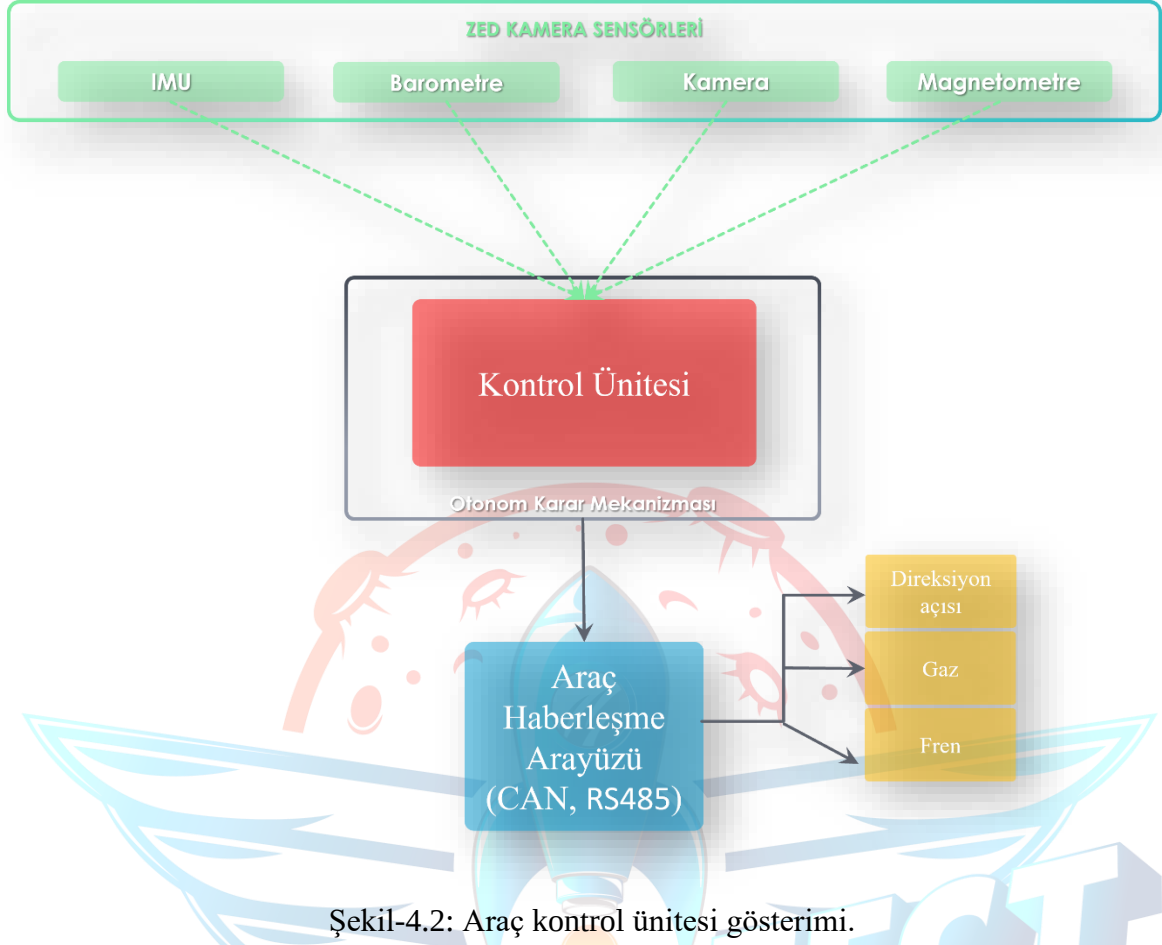
Hazır araç kategorisi kapsamında kullanımımıza sunulan aracın otonom kontrol ünitesi şekil-3.3’de görüldüğü gibi tasarlanmıştır. Bu kapsamda sensörlerden alınan bilgilerin işlenmesi ve anlamlandırılması için bilgisayar veya gömülü donanımlar kullanılabilir. Hazır araç kapsamında ekibimize otonom araç çalışmaları özelinde geliştirilmiş Nvidia Xavier AGX 32 GB geliştirme kartı sağlanmıştır. Bu kapsamda Xavier geliştirme kiti şekil-4.1’deki gibidir. Bu kitin özellikleri tablo-4.1 ve sistem ile olan entegrasyonu da şekil-4.2 de ki gibidir.



Şekil-4.1: Xavier AGX kart görünümü. [16]

GPU	Volta GPU with Tensor Cores 8-core ARM v8.2 64-bit CPU
Depolama	32GB eMMC 5.1
DL Accelerator	2x-NVDLA Engines
Vision Accelerator	7-way VLIW Vision Processor
Encoder/Decoder	2x-4Kp60 HEVC-2x 4Kp60 12-Bit Support
Size	105mm x 105mm
PCIe X16	X8 PCIe Gen4/x8 SLVS-EC
RJ45	Gigabit Ethernet
USB-C	2x USB3.1
Kamera Bağlantısı	(16x) CSI-2 Lanes
M.2 Key M	NVMe
40-Pin Header	UART + SPI + CAN + I2C + DMIC + GPIOs

Tablo-4.1: Xavier AGX Geliştirme Kit Özellikleri. [16]



Şekil-4.2: Araç kontrol ünitesi gösterimi.

Araç, kablo ile sürüşe (by-wire) sahip ve bu sistemlerde CANbus iletişim yolu üzerinden haberleşmekte olan elektrikli bir araçtır. Araç kontrol ünitesinde genel olarak Velodyne Lidar, ZED 2 Kamera, RF Anten, By-Wire sistemleri ve Nvidia Xavier AGX kiti bulunmaktadır.

Jetson AGX Xavier 32 GB, 20 Watt altında çalışma gücüne sahiptir. Jetson AGX Xavier derin öğrenme, bilgisayar görüşü, GPU hesaplama, multimedya işleme ve daha fazlası için board support package (BSP), Linux OS, NVIDIA CUDA ve cuDNN yazılım kütüphanelerini içeren NVIDIA JetPack SDK tarafından da desteklenmektedir. Bu bağlamda destek sağlanıyor olması geliştirmeler için kolaylıklar sağlamaktadır.

5. Araç Dinamiği Modelleme/Tanılama ve Kontrolü

Aracın dinamiğine dair genel görünüm ve özellikleri şekil-3.1 de ki gibidir. Bu bağlamda yer alan araç özelliklerine göre geliştirilmiş olan hareket ve kontrol sistemleri aşağıdaki gibidir.

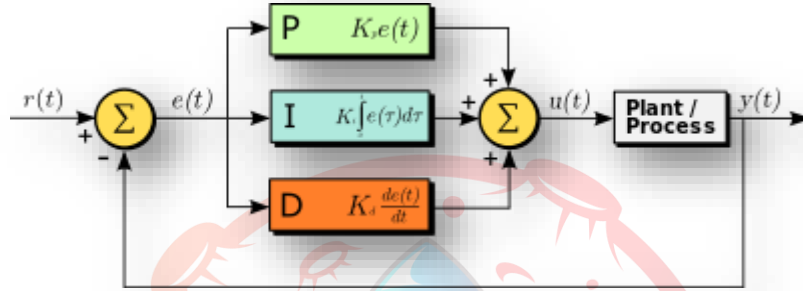
5.1 – İleri Beslemeli Açık Kontrolü

Şerit algoritmasından elde edilen verilere göre aracın dönüş açısının kontrolü için PID kontrolü kullanılmaktadır. Endüstriyel kontrol sistemlerinde en çok kullanılan kontrollerden olan PID kontrolü, geri besleme yoluyla hata farkı (sistemin mevcut durumu ve istenen durumu arasındaki fark)'na göre çıkışa bir sinyal üreterek ilgili sistemlerin kontrolünü sağlayan bir kontrol yöntemidir. Proportional Integral Derivative yani Oransal İntegral Türevsel açılımı olan PID, P(Oransal), I (İntegral) ve D (Türevsel) olmak üzere 3 tane farklı parametreye sahiptir. Bu

parametrelerden P mevcut hata, I geçmiş hataların toplamı, D gelecekteki hataların bir tahminini ifade eder. PID kontrol yönteminin zaman domainindeki matematiksel ifade ve blok diyagramı aşağıdaki gibidir.

$$K_p e + K_d \frac{de}{dt} + K_i \int_0^t e(t) dt$$

Şekil-5.1.1: PID Blok Diyagramı [1]



Şekil-5.1.2: PID Blok Diyagramı [2]

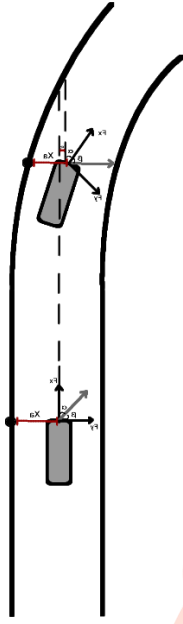
Burada K_p , K_d ve K_i olarak gösterilen katsayılar P, I ve D parametrelerin katsayılarıdır. Bu katsayıları farklı algoritmaları kullanılarak en iyi sonuç elde edilebildiği gibi farklı denemelerin sonucunda da elde edilebilmektedir. Farklı denemelerde $K_p = 1.0 / 835.0$, $K_i = 1.0 / 600.0$, $K_d = 1.0 / 1000.0$, değerleriyle en iyi sonuçlar elde edilmiştir.

Burada istenilen değer araç haberleşme arayüzü CAN ile araca gönderilirken aracın mevcut dönüş açısını ZED 2 kamerada yer alan IMU sensörünün (Inertial Measurement Units - **İnersiyal Ölçme Ünitesi**) verilerinden elde etmekteyiz. ZED 2 Kamerasının IMU sensöründen anlık olarak (“Parent” – Araç) açısal hız, açısal ivme ve oryantasyonu gibi bilgiler sağlanmaktadır.

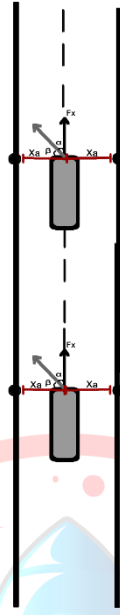
PID Hesaplanması sözde kodu aşağıdaki şekildedir.

```
errorSum = Error + errorSum;      # Baştan şu ana kadarki hataların toplamı alındı
errorDif = errorPre - Error;      # Bir önceki ve şimdiki hataların farkı alındı
errorPre = Error;                 # Bir önceki hata güncellendi

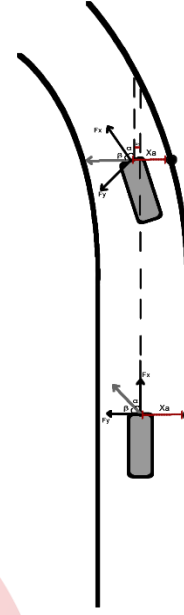
P = Kp*Error;                     # Oransal Kontrol Sinyali hesaplandı
I = Ki*errorSum*dt;               # Integral Kontrol Sinyali hesaplandı
D = Kd*errorDif/dt;              #Türev Kontrol Sinyali hesaplandı
PID = P + I + D;                  # O,I ve T kontrol sinyallerinin toplamıyla
                                  # PID Kontrol sinyalini elde ettik
```



Şekil-5.1.3: Sol şerit takibi



Şekil-5.1.4: Düz şerit takibi



Şekil-5.1.5: Sağ şerit takibi

Şekil-5.1.3, şekil-5.1.4 ve şekil-5.1.5 de yer alan görsellerde düz, sağ ve sol açı kontrolleri yer almaktadır.

5.2 – CAN ile haberleşme

Sahip olduğumuz araç ile haberleşme CANbus protokolü üzerinden sağlanmaktadır. Aracın kontrolü için kullanılan parametrelerin teknik bilgileri şu şekildedir;

operational:	Sistemi otonom olarak sürmek için devreye almayı sağlamaktadır. 0= OFF, 1=ON
steeringAngle	Aracın direksiyonunu sağa sola çevirme komutudur. -128 ile +128 arasında değişmektedir. -128 sola tam dönüş, +128 sağa tam dönüştür.
speed	Aracın ileri ve geri gidiş komutudur. -128 ile +128 arasında değişmektedir. -128 geri vites maksimum hız, ileri vites maksimum hız mesajıdır
brake	Araca fren yaptıran mesajdır. 0 ile 255 arasında değişmektedir. Acil durdurma maksimum fren 255 değeridir.

Tablo-5.1.1: Araç kontrolü için kullanılan parametreler.

CAN Mesaj ID:	0x560										
Name	Message	Multiplexir	Startbit	Length [Bit]	Byte Order	Value Type	Initial Value	Factor	Offset	Minimum	Maximum
operational	VEHICLECTRL	-	0	1	Intel	Unsigned	0	1	0	0	0
steeringAngle	VEHICLECTRL	-	8	8	Intel	Unsigned	0	1	-128	-128	127
speed	VEHICLECTRL	-	16	8	Intel	Unsigned	0	1	-128	-128	127
brake	VEHICLECTRL	-	24	8	Intel	Unsigned	0	1	0	0	0

Şekil-5.1.1: Araç kontrolü can üzerinden gönderilen byte aralıkları ve parametreleri.

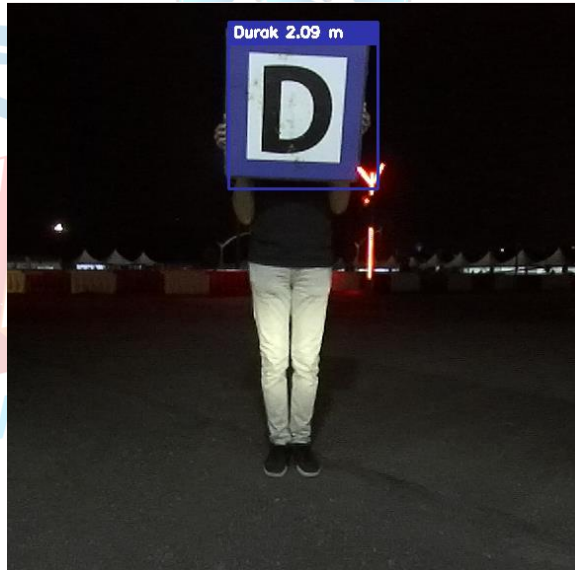
5.2 – ZED 2 Kamera ile Mesafe Kontrolü

Aracın sahip olduğu ZED 2 kamerasının hareket/çevre sensörlerinin özellikleri şekil-5.2.1 de ki gibidir.

```
Camera Model: ZED 2
Serial Number: 22784739
Camera Firmware: 1523
Sensors Firmware: 1523
*****
Sensor type: SENSOR_TYPE.ACCELEROMETER
Max rate: 400.0 SENSORS_UNIT.HERTZ
Range: [-78.48000336 78.48000336] SENSORS_UNIT.M_SEC_2
Resolution: 0.002395019633695483 SENSORS_UNIT.M_SEC_2
Noise Density: 0.0015999999595806003 SENSORS_UNIT.M_SEC_2/√Hz
Random Walk: 0.0002508999896235764 SENSORS_UNIT.M_SEC_2/s/√Hz
*****
Sensor type: SENSOR_TYPE.GYROSCOPE
Max rate: 400.0 SENSORS_UNIT.HERTZ
Range: [-1000. 1000.] SENSORS_UNIT.DEG_SEC
Resolution: 0.030517578125 SENSORS_UNIT.DEG_SEC
Noise Density: 0.007000000216066837 SENSORS_UNIT.DEG_SEC/√Hz
Random Walk: 0.0019474000437185168 SENSORS_UNIT.DEG_SEC/s/√Hz
*****
Sensor type: SENSOR_TYPE.MAGNETOMETER
Max rate: 50.0 SENSORS_UNIT.HERTZ
Range: [-2500. 2500.] SENSORS_UNIT.U_T
Resolution: 0.30000001192092896 SENSORS_UNIT.U_T
Noise Density: 0.07000000029802322 SENSORS_UNIT.U_T/√Hz
*****
Sensor type: SENSOR_TYPE.BAROMETER
Max rate: 25.0 SENSORS_UNIT.HERTZ
Range: [ 300. 1100.] SENSORS_UNIT.HPA
Resolution: 0.009999999776482582 SENSORS_UNIT.HPA
Noise Density: 0.00039999998989515007 SENSORS_UNIT.HPA/√Hz
```

Şekil-5.2.1: ZED 2 kamerasının sahip olduğu hareket/çevre sensörlerinin özellikleri.

Kameranin sağlamış olduğu derinlik özelliği sayesinde trafik tabela ve ışıklarının konumlarının erkenden tespit edilmesi şekil-5.2.2 de ki gibi sağlanabilmektedir.



Şekil-5.2.2: ZED 2 kamerasının derinlik özelliği sayesinde tespit ettiğimiz tabela konumu.

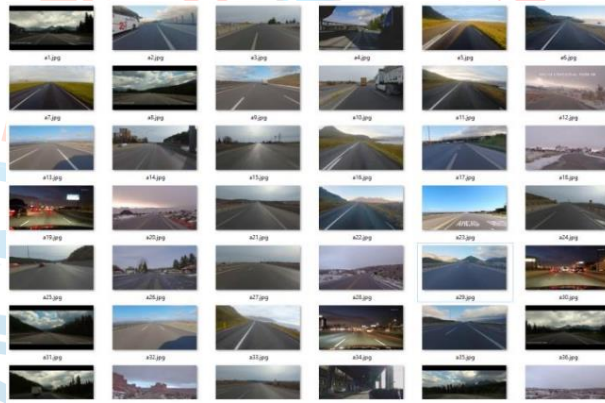
6. Otonom Sürüş Algoritmaları

Göbeklitepe OSAT takımı tarafından yarışma kapsamında aşağıda verilen otonom sürüş algoritmaları geliştirilmiştir.

6.1 - Şerit/Yol Çizgisi Tanıma Modülü

1-) Şerit takibi için mevcut algoritmaların birçoğu araştırılmış ve bu algoritmaların eksiklikleri ile olumsuz etki uyandıran yönleri tespit edilmiştir. Bu tespitler göz önüne alınarak şerit tanıma için özellikle FPS anlamında daha performanslı sonuçlar döndürdüğünden YOLOv4-Tiny modeli Darknet ortamında kullanılmıştır.

2-) Github platformunda yayınlanan, dahil olduğumuz verisetlerinden ve yarışma alanından çekmiş olduğumuz 24 farklı videodan aralıklı fotoğraflar olarak yaklaşık 4000 adet fotoğraf verisi oluşturuldu.



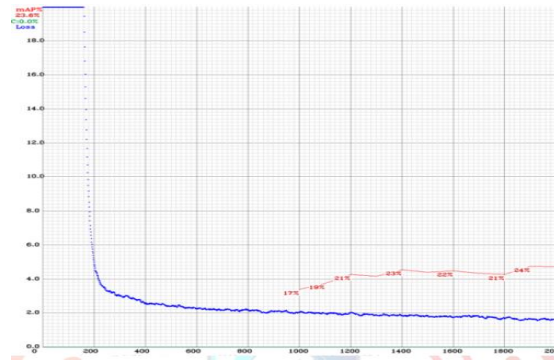
Şekil-6.1.1: Şerit tanıma için kullanılan veri setlerimiz.

3-) Toplanan görselleri etiketlemek için makesense.ai web sitesini kullandık. Bu etiketleme aracı ile şeritleri yol boyunca etiketleyerek her fotoğrafın etiket dosyasını oluşturduk.



Şekil-6.1.1: Görseller üzerinde yapılan etiketlemelere yönelik görsel.

4-) Hazırlanan veri setini %20 test ve %80 eğitim verisi olarak ayırarak eğitime hazır hale getirdik. Veri setini eğitmek için darknet sistemi ve eğitim platformu olarak da Google Colab kullandık. Eğitim grafiğimiz ve değerlerimiz aşağıdaki görselde mevcuttur.

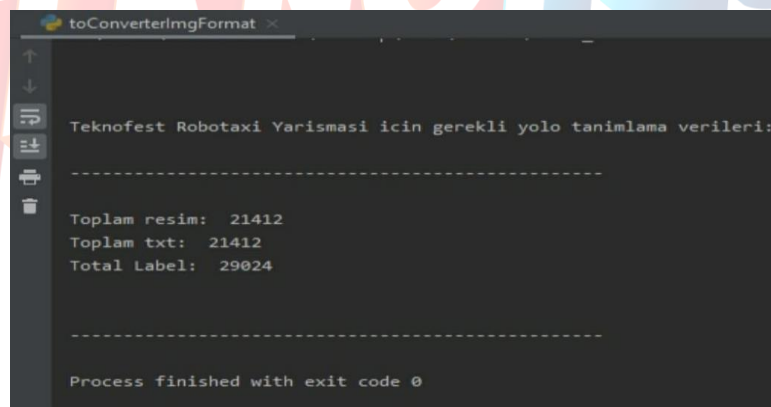


5-) Çıkan modeli birçok videoda test ederek algılama yeteneğini ve FPS durumunu gözlemledik. FPS değerleri yaklaşık olarak 60 - 65 FPS olarak değişmektedir. Optimizasyon çalışmalarıyla bu değer daha yüksek seviyeye getirilecektir.

6.2 - Levha/Trafik Işığı Tanıma Modülü

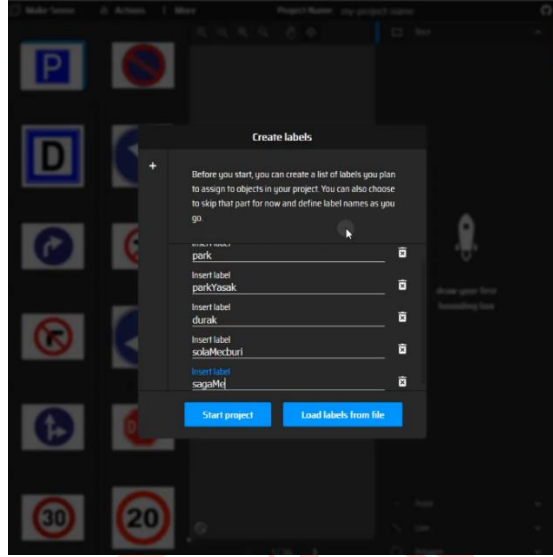
1-) Levhalarımızı algılayabilecek yapay zekâ modelleri hakkında araştırmalar yapıldı. Araştırmalarımız sonucunda modellerin avantajları ve dezavantajlarını gözlemleyerek modellerin sistem gereksinimleri, saniye başına düşen kare sayısı (FPS) ve doğruluk değerleri gibi temel konulara çözüm bulan YOLOv4-Tiny modelinin darknet ortamında kullanmanın en doğru seçim olacağına karar verildi.

2-) Tabelalarımızın çeşitli programlar aracılığıyla farklı açılardan fotoğrafları çekildi ve amacımıza uygun olan veri setlerine erişim sağlayarak yüksek yoğunluklu bir veri seti oluşturuldu. Bu veri setimizi desteklemek amacıyla yarışma alanında yapmış olduğumuz testlerde levhaların videoları çekilerek kareler elde edildi ve böylelikle veri setimiz 17 tabela için 21500 fotoğraftan oluşturulmuş oldu.



Şekil-6.2.2.1: Trafik tabelası ve ışığı veri seti ile tabela başına düşen veri adet görseli.

3-) 21500 fotoğraftan oluşan veri setimizin etiketleme işlemi “makesense.ai” aracı kullanılarak gerçekleştirildi. Bu işlem sonucunda 32074 tane etiketlenmiş veri elde ettik.



Şekil-6.2.3.1: Etiketleme için kullanılan makesense.ai ortamı.

4-) Veri setlerimizi %80 eğitim, %20 test olarak hazırladık ve model eğitimi işlemini gerçekleştirdik. Eğitim sonucunda elde etmiş olduğumuz modelin simülasyon üzerinde gerçek zamanlı testleri gerçekleştirilmiş ve başarılı bir şekilde sonuç verdiği gözlemlenmiştir.

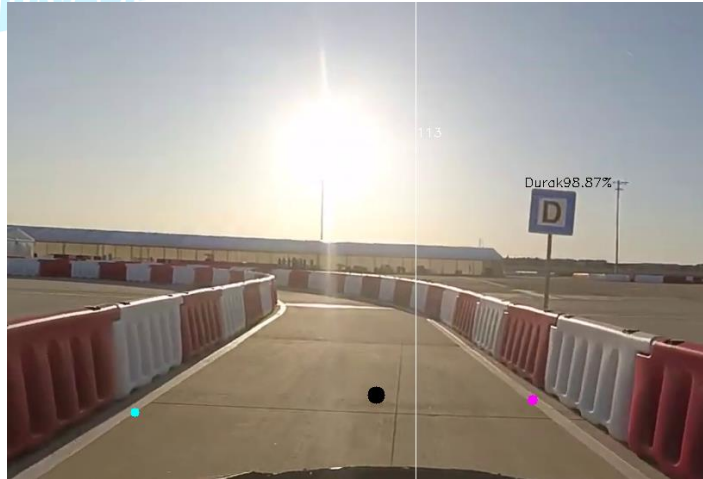
```
for conf_thresh = 0.25, precision = 0.88, recall = 0.84, F1-score = 0.86
for conf_thresh = 0.25, TP = 1179, FP = 157, FN = 219, average IoU = 71.05 %

IoU threshold = 50 %, used Area-Under-Curve for each unique Recall
mean average precision (mAP@0.50) = 0.313428, or 31.34 %
Total Detection Time: 59 Seconds
```

Şekil-6.2.4.1: Eğitilen modelin test sonuçları.

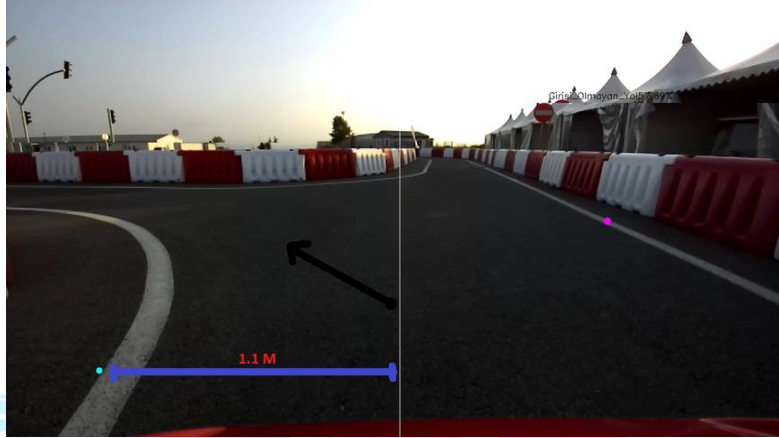
6.3 - Sürüş Algoritma Modülü

Şerit tanıma modülünden alınan şerit verileri çok fazla olduğundan hangi şeritin sağ tarafa ait olduğu hangilerinin ise sol tarafa ait olduğu tam olarak bilinmemektedir. Bu noktalar sağ ve sol olmak üzere iki gruptan oluşturulmuştur. Gruplama yapılırken sağdaki ve soldaki gruplanmış noktalardan birbirine yakın olanların ortalaması alınmış, sağda ve solda tek bir nokta kalana kadar birleştirilmiştir.



Şekil-6.3.1: Yarış parkuru şerit ve tabela tespiti testi.

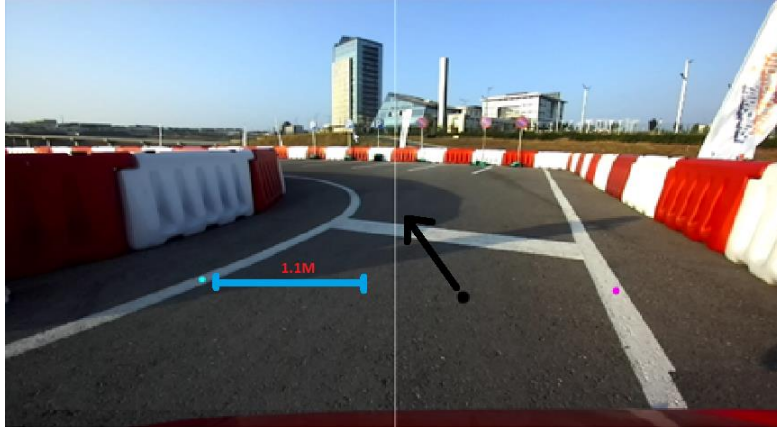
Dönüş açısı bulabilmek için sağda ve solda oluşan noktaların ikisi arasındaki orta nokta şekil-6.3.2 de ki gibi tespit edilir. Bu değer zed kameradan alınan framerin orta noktasına olan uzaklığından çıkartılarak bir dönüş değeri elde edilir. Ayrıca aracın dönüş aldığı veya ekranda yalnızca bir tarafın şeriti tespit edilmişse tespit edilen şeritin takip edebilmesini sağlayabilmek gerekmektedir. Herhangi bir şerite aracın kendisini belirli bir uzaklıkta tutmasını sağlamak için bir dönüş değeri elde edilir. Motor ve donanım iletişimini CANbus haberleşme protokolü ile yapacağımızdan dolayı bu değeri can protokolüne uygun hale getirmemiz gerekmektedir. Bulmuş olduğumuz dönüş açısı değeri tekerin dönüş açısına yani 0 ile 255 arasında bir değere dönüştürülmüştür. CANbus protokolü ile bu değer aracın tekerlerine iletilir.



Şekil 6.3.2: Aracın tabela kurallarına uyarak belirli mesafede şerit takibi yapma görseli.

Sürüş algoritmamız dört temel üzerine kurulmuştur. ZED 2 marka kameradan IMU, görüntü ve derinlik alma, direksiyon kontrolü için şerit tanımlama, karar verme modülü için levha tanımlama ve bu yapmış olduğumuz algoritmalarından alınan verilerin kontrol algoritmasından geçirilerek CAN mesajı gönderme şeklinde geliştirilmiştir. Trafik levha ve ışıklarına yönelik geliştirilen bazı adımlar aşağıdaki gibidir.

- Araç trafik levhalarından durak levhasını algıladıktan sonra ZED kameranın derinlik özelliği kullanılır. 1 metre kalana kadar ilerlenir, sonrasında fren değeri kademeli olarak 255'e getirilir. Hız değeri 128 olarak ayarlanır, 30 saniye bekletilerek aynı işlemlerin tersi yapılarak aracın devam etmesi sağlanır.
- Trafik ışıklarında kırmızı ışık algılandığı süre boyunca can mesajında fren değeri 255'e getirilerek aracın durması sağlanır, yeşil olduğunda ise araç fren değerini kademeli olarak 0 değerine getirir ayrıca hız değeri de kademeli olarak artırılır.
- İleri ve sola, ileri ve sağa, sola, sağa, tabelalarında sola dönülmesi gerekiyorsa sol şeridi referans alınarak sol şerit takip edilir. Sağa dönülmesi gerekiyorsa sağ şerit referans alınarak sağ şerit takip edilir.
- Sola dönülmez veya sağa dönülmez tabelalarında ise bunu tam tersi olarak, sağa dönülmez tabelası varsa sol şeridi, sola dönülmez tabelası varsa sağ şeride göre araç kendini sabitler ve o şerit takip edilir.
- Bitiş çizgisi tespit edildikten sonra park alanı sol da ise sol şerit takip edilerek aracın sola dönmesi sağlanır. Park edilebilir levhasına kadar sabit hızla devam eder. Park tabelası görüldüğünde ise araç tabelaya doğru dönüş alır ve ekranın orta kısmında algılanan şeritlere göre kendini ortalar böylece araç kendini park etmiş olur.



Şekil 6.3.3: Park alanı giriş görseli.

7. Güvenlik Önlemleri

Test aşamasında ve yarışma sırasında olası tehlikeli durumlar için alınması gereken 4 farklı güvenlik sistemi belirlenmiştir. Bu sistemler ve sistemlere yönelik alınması planlanan güvenlik önlemleri şu şekildedir.

7.1 - Uzaktan Acil Müdahale Yöntemleri

Hazır araç ile birlikte verilen kumanda sayesinde araca acil bir müdahalede bulunulması gerekirse bu kumanda sayesinde müdahale edebileceğiz. Can komutları kumandadan araca 20 ms'lik aralıklarla gönderilir. Biz ise kod tarafında 30 ms'lik aralıklarla can komutlarımızı göndeririz. Bu sayede kumanda daha hızlı mesaj gönderdiğinden acil bir müdahalede aracı durdurabiliyoruz.

7.2 - Fiziksel Acil Müdahale Yöntemleri

Aracın acil durumlarda frenleme yapmasının yanında elektrik bağlantısının da acil olarak kapatılması gerekir. Bu durum için aracın üzerinde bulunan acil durum butonları bu görevi yerine getirmektedir.



Şekil-7.2.1: Acil müdahale butonu.

7.3 – Yazılımsal Acil Müdahale Yöntemleri

Aracın kendisine ve çevresine zarar vermemesi adına yazılımsal olarak bazı önlemler aldık. Bu önlemlerden ilki zed kameradan aldığımız verilerden derinlik verisi sayesinde aracın sağdaki ve soldaki bariyerlere uzaklığını anlık olarak alıp bu uzaklık değerinin belirli bir değerin altına düşmesi halinde fren ve hız değerlerini değiştirerek durmasını sağladık.

7.4 - Hız Kontrol Güvenlik Önlemleri

Görev isterlerinin başarılı bir şekilde tamamlanması için aracımızın görevlerini yerine getirirken aynı zamanda olabildiğince hızlı hareket etmesi gerekmektedir. Bu doğrultuda bakıldığında aracın istenilen seviyeye güvenli bir şekilde çıkması ve istenilen seviyede hızını tutabilmesine yönelik bir hız kontrol görevi geliştirilmiştir.



8. Simülasyon

İşletim Sistemi

İşletim sistemi **Ubuntu** 18.04 versiyonudur.

- Ubuntu 18.04 işletim sistemi Açık kaynaklı bir yazılım olması ve kullanılan donanımın nvidia xaviera ubuntu sürümünün 18.04 desteklemesidir.

Simülasyon ortamı geliştirme programları

Harita Geliştirme programlar için Sketchup, Blender kullanılmaktadır.

- Simülasyon ortamı haritası için çizim ve animasyon için Sketchup ve Blender kullanıldı. Haritanın Araba ile entegrasyonu ve Algoritmalar kullanmak için Gazebo ortamında geliştirilmiştir

ROS

Ros Noetic versiyonu kullanılmıştır.

- Açık kaynaklı robotik programlama ara yüzüdür.
- Robot uygulamaları geliştirmeleri için gerekli kütüphaneleri ve araçları sağlar.
- Teknofest tarafından gönderilen araç modelin desteklediği ROS sürümüdür.

CUDA

CUDA 10.2. sürümünü kullanmaktayız.

- Yapay zekâ modelimizi ekran kartı üzerinde çalıştırarak paralel programlama ile işlem hızımızı artırır ve işlem ünitesini en verimli hale getirilmesini sağlar.

YOLO

YOLOv4-tiny versiyonu kullanılmıştır.

- Gerçek zamanlı bir nesne algılama modelidir. Levha ve şerit modelleri büyük katmanını tanıma oranı ve çalışma hızı bakımından YOLOv4-tiny kullanılmıştır.
- YOLOv4-tiny 2 katmanlı (Büyük, Orta) hafif bir yapısı bulunduğu için kullanılmıştır.

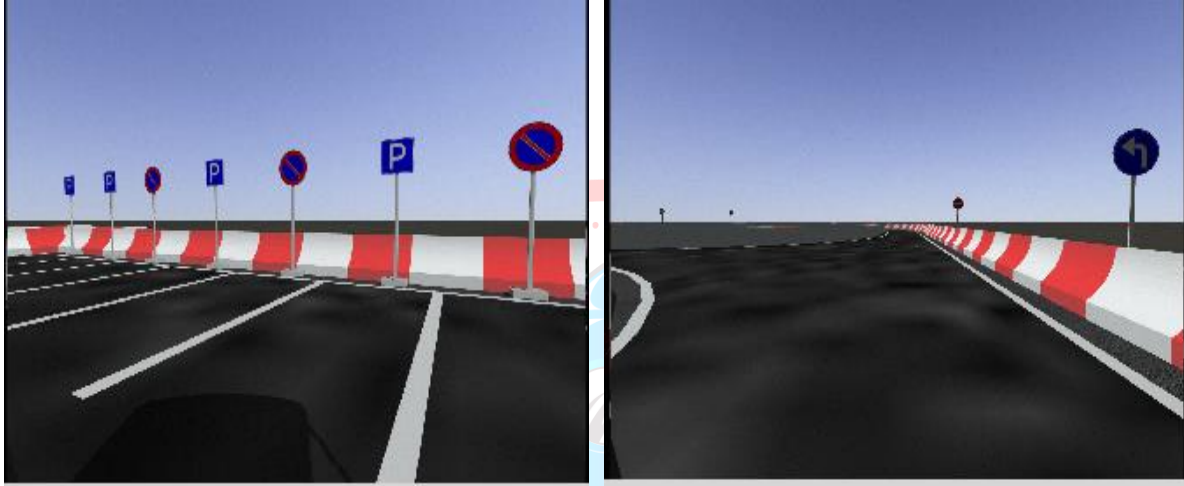
Simülasyon Motoru

Gazebo9 kullanılmıştır.

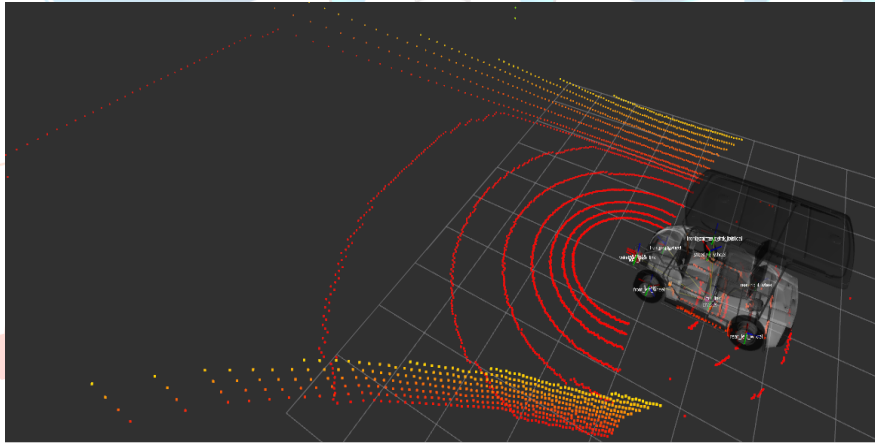
- Verilen Araç modeline uygun bir simülasyon motoru olduğundan tasarlanan harita Gazebo9 ortama aktarılmıştır
- Simülasyon ortamı olarak Gazebo9 kullanılmıştır.

Belirlenen ölçülerde çizilen harita yol uzunlukları ve yol genişlikleri şartnamede belirtilen ölçülere göre ayarlanmıştır ve simülasyon ortamına uygun şekilde çizilmiştir. Harita Ubuntu 18.04 işletim sisteminde Blender programında animasyon işlemleri bittikten sonra .dae uzantılı formatta dışarıya aktarılmıştır. Gazebo simülasyon ortamı .dae formatta dosyaları kabul ettiğinden simülasyon haritası bu uzantıda aktarılmıştır ve düzenlemeler yapılmıştır.

Simülasyon ortamı roslaunch cart_sim cart_sim.launch yapıldığı zaman harita ortamı GOLF arabası ile birlikte harita yüklenmektedir. Rviz ile Gazebo ortamında harita görülmektedir.

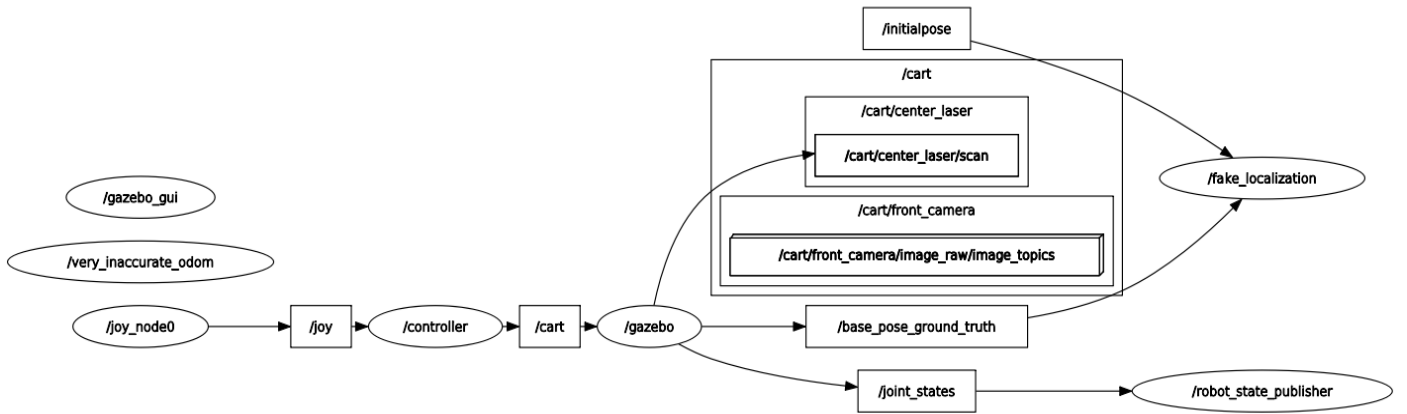


Şekil-8.2: Rviz-Simülasyon ortamı görseli.



Şekil-8.3: Rviz-Araç Sensor Görseli.

8.1. Simülasyon Mimarisi



Şekil-8.1.1: Simülasyon mimarisi

8.2. Simülasyonda Kullanılan Algoritmalar

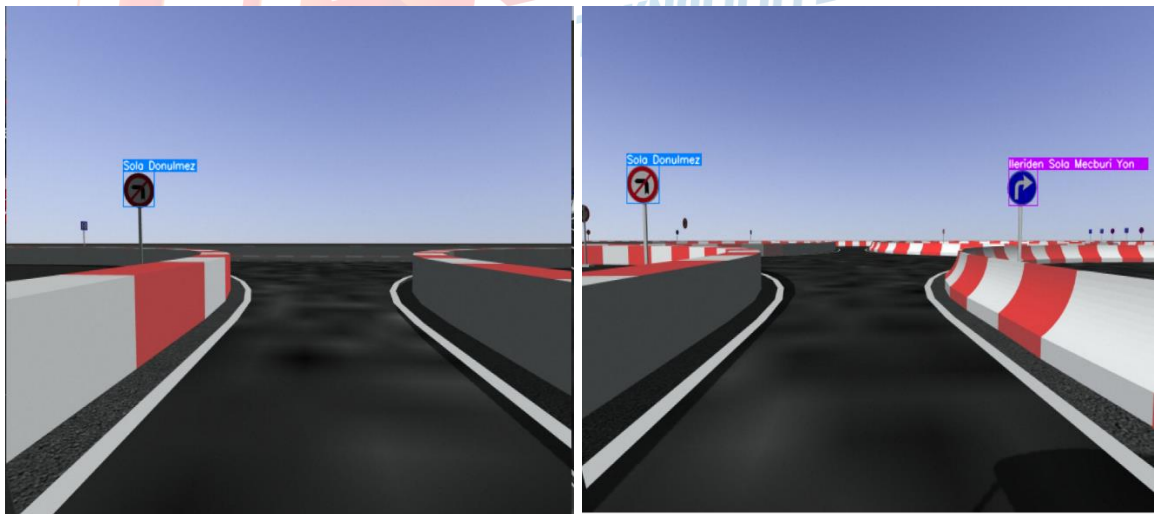
8.2.1. Model algoritması

Gazebo 'ya .dae formatında aktarılan haritanın modeli 'uri' ile içeri aktaran ve konumu, boyutu ve materyali gibi harita 'ya ait verilerin bulunduğu XML algoritmasıdır.

8.2.2. Şerit ve Trafik levhaları tanıma algoritması

Simülasyon ortamında elde edilen görüntüler üzerinde model eğitildi ve eğitilen model şerit ve trafik tanımak için YOLOv4-tiny nesne tanıma algoritması kullanıldı.

Şerit ve Trafik levhaları için 2 farklı model eğitildi. Bu modellerde Araç hareket ederken hem şerit ve hem Trafik levhalarını tanımaktadır ve bu tabelaların durumuna göre araç tepki vermektedir.

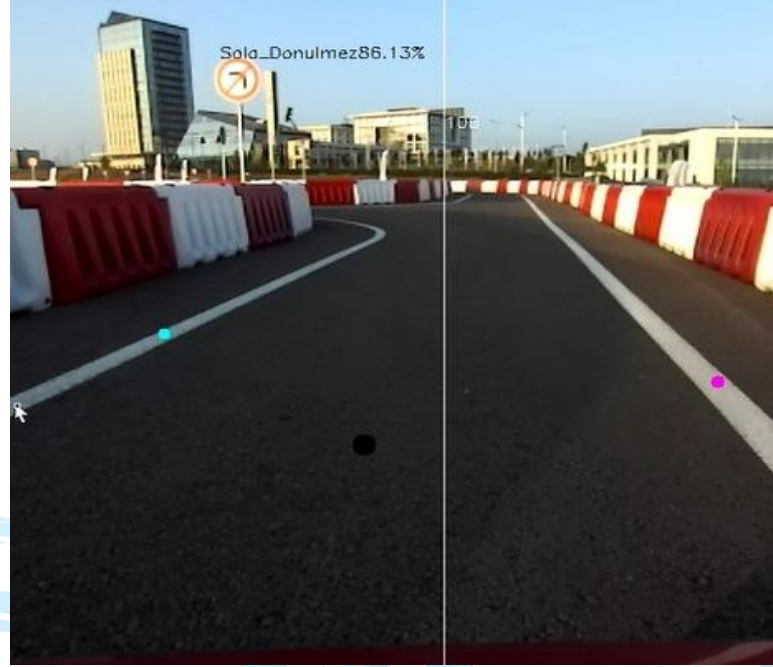


Şekil-8.2.1: Şerit ve Tabela okuma

Yukarıdaki görselde görüldüğü gibi araç sağ ve sol şeritleri takip ederek kendini kontrol

etmektedir. Bu kontrol sonucunda araç bir yere çarpmadan hareket edebilmektedir. Araç giderken eğitilen model sayesinde Şekil-8.2.1’de gibi tabelaları okuyabilmektedir.

Eğitilen model ve kullanılan algoritmalar gerekli düzenlemeler sonucunda gerçek hayatta test edilmiş ve aynı sonuçlar başarılı bir şekilde elde edilmiştir.



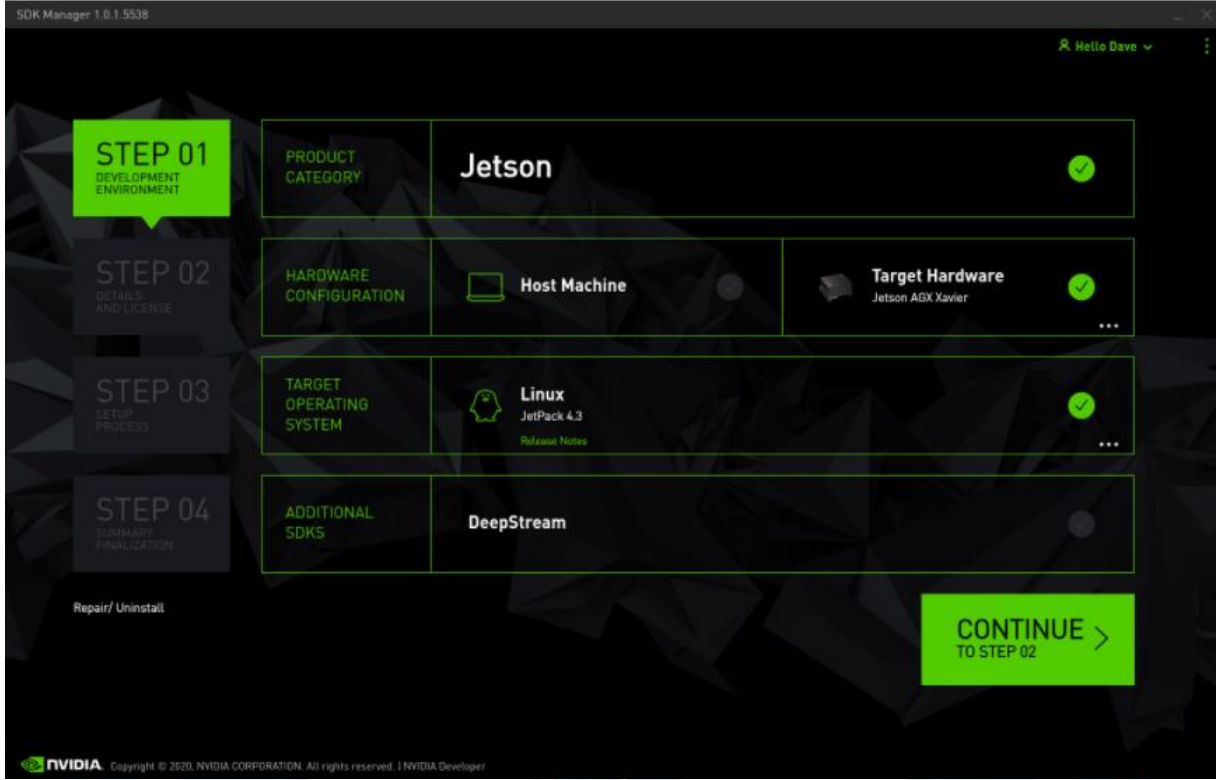
Şekil-8.2.2: Parkur alanından test görüntüsü.

9. Sistem Entegrasyonu

Hazır araç içerisinde Nvidia Jetson AGX Xavier kiti tercih edilmiştir. AGX Xavier kiti otonom makineler için geliştirilmiş 30W'ın altındaki bir modül üzerinde GPU iş istasyonu performansını sunduğu bir yapay zeka bilgisayardır. CAN-BUS haberleşme protokolünü destekler ve üç tane pwm sinyal yolu barındırır. Aynı zamanda kurulum işlemleri Nvidia SDK tool arayüzü üzerinden gerçekleştirilebilir. Cihaz üzerinde Jetpack 4.4 kurulumunu tercih ettik. Kurulum tamamlandıktan sonra CANbus haberleşme üzerine kurulumları tamamladık.

9.1. Xavier Sistem Entegrasyon Adımları

Nvidia Jetson Xavier AGX kurulumu için gerekli olan SDK Manager kurulumu Nvidia sayfasına üye olunarak elde edilir. Kurulum tamamlandıktan sonra Ubuntu 18.04 ile cihaz arasında kablolu bağlantılı sağlanır. Sonrasında şekil-9.1.1 de görünen Target Hardware bölümünde cihazın algılanması beklenir.



Şekil-9.1.1: SDK Manager Arayüzü [17]

Sonrasında Xavier kurulumu için gerekli olan önemli kütüphaneler indirilmeye başlanmaktadır. Bu bağlamda kurulan paketler şekil-9.1.2 de ki gibidir. Bu adım sonrasında kurulumlar sırasıyla devam etmektedir.



Şekil-9.1.2: Xavier kurulumu sırasında kurulan paketler [17]

Ardından klasik ubuntu 18.04 kurulumu gerçekleştirilmektedir. Bu kurulum sonrasında şekil-9.1.3 de ki gibi bir görüntü elde edilmektedir.



Şekil-9.1.3: Xavier Ubuntu 18.04 masaüstü arayüzü. [17]

9.2. CAN-BUS Haberleşme Arayüzü Kurulumu

Aracın gazını ve frenini kontrol etmek için gereken 0-10V arasındaki voltaj değerleri, kontrol ünitesi üzerindeki PWM kanalları ile kontrol edilir. PWM kanallarını kullanma CAN-BUS haberleşme formatında olduğu gibi pinmux değişikliklerinin yapılmalıdır. Öncelikle python üzerinden kurulumu gerçekleştirebilmek için gerekli olan python-can ve busybox kütüphanesi kurulmaktadır. Sonrasında sistemin CAN kontrolörlerinin hangi pinlere karşılık geldiğini yapılandırıyoruz. Bu adımlar ve gerekli tanımlamalar için aşağıdaki adımları takip ediyoruz. Ayrıca tanımlama yapılması gereken pin karşılıkları şekil-9.2.1 de ki gibidir.

Canbus Aktifleştirme

- Busybox isimindeki paketi kurarak xavier üzerindeki kayıt değerleri değiştirilmelidir.
- CAN kontrolörlerinin pinlerine karşılık gelen dört kayıt değerini yeniden yapılandırmak gerektiğinden aşağıdaki komutlar girilerek pinler yeniden yapılandırılır.
 - `sudo busybox devmem 0x0c303000 32 0x0000C400`
 - `sudo busybox devmem 0x0c303008 32 0x0000C458`
 - `sudo busybox devmem 0x0c303010 32 0x0000C400`
 - `sudo busybox devmem 0x0c303018 32 0x0000C458`
- Aşağıdaki komutlar ile gerekli sürücüler yükliyoruz
 - `sudo modprobe can`
 - `sudo modprobe can_raw`
 - `sudo modprobe mttcan`

- CAN Kontrolörlerini CAN veri yolu ve alıcı-vericilerinizin özelliklerine göre yapılandırıyoruz.
 - `sudo ip link set can0 type can bitrate 500000 \`
 - `dbitrte 2000000 berr-reporting on fd on`
 - `sudo ip link set can1 type can bitrate 500000 \`
 - `dbitrte 2000000 berr-reporting on fd on`
- Şimdi sadece CAN arayüzlerini açmamız gerekiyor:
 - `sudo ip link set up can0`
 - `sudo ip link set up can1`
- Yapılan bu can yapılandırmalarının sistem başladığında otomatik yapılması için `enable_CAN.sh` isimli dosya oluşturup içerisine yukarıdaki kodlar eklenerek kök dizine eklenebilir bu sayede sistem her açıldığında otomatik olarak yapılandırma yapılmış olur.
 - `printf '%s\n' '#!/bin/bash' 'exit 0' | sudo tee -a /etc/rc.local`
 - `sudo chmod +x /etc/rc.local`
- Bu işlemler sonunda `enable_CAN.sh` dosyası sistem başlangıcında otomatik olarak çalıştırılır ve can denetleyicilerini yapılandırır.

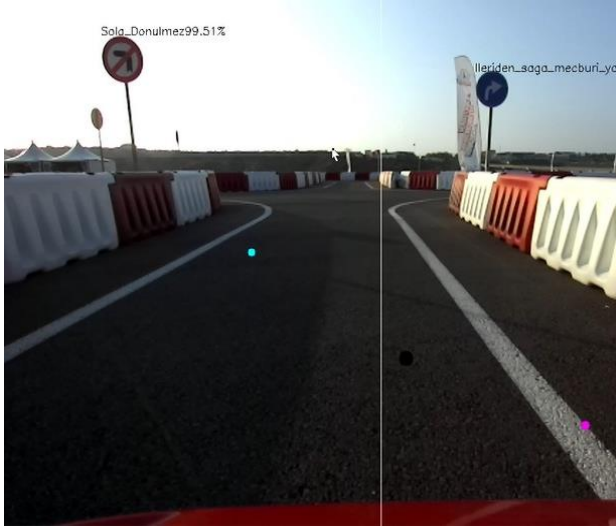
3.3V	1	2	5.0V
I2C_GPS_DAT	3	4	5.0V
I2C_GPS_CLK	5	6	GND
MCLK95	7	8	UART1_TX
GND	9	10	UART1_RX
UART1_RTS	11	12	I2S2_CLK
PWM01	13	14	GND
GPIO27_PWM2	15	16	GPIO8_AO_DMIC_IN_DAT
3.3V	17	18	GPIO35_PWM3
SPI1_MOSI	19	20	GND
SPI1_MISO	21	22	GPIO17_40HEADER
SPI1_SCK	23	24	SPI1_CS0
GND	25	26	SPI1_CS1
I2C_GP2_DAT	27	28	I2C_GP2_CLK
CAN0_DIN	29	30	GND
CAN0_DOUT	31	32	GPIO9_CAN1_GPIO0_DMIC_CLK
CAN1_DOUT	33	34	GND
I2S_FS	35	36	UART1_CTS
CAN1_DIN	37	38	I2S_SDIN
GND	39	40	I2S_SDOUT



Şekil-9.2.1: Xavier pinlerine karşılık gelen değerler. [18]

10. Test ve Doğrulama

Aracımızda yapılan testlerde Trafik işaretlerini okuma ve şerit takibi algoritmalarında YOLOv4-tiny kullanılarak modeller eğitilmiştir. Tabela tanımlama aşamasında yapılan tasarımda %90 ve üzeri doğru tahmin etme istatistiği tespit edilmiştir. Saha testlerinde Araç'a uygulanan teste algoritma üzerinde filtreleme Güneş ışıklarına karşı görüntünün daha derlenebilir bir halde elde edilmesinden Tabela tanımda görüntülerde bu oran birbirine yakın değerler gelmiştir.



Şekil-10.1: Gerçek ortam



Şekil-10.2: Simülasyon ortamı

Şekil-10.1 ve Şekil-10.2 'de aynı anda çekilmiş görüntülerde Tabelaların doğruluğunu ve gerçek ortamda bu doğruluğun yüzdesi gözükmemektedir. Şerit Tanıma Modülünde simülasyon ortamında yapılan testler sonucunda aracın şerit ve tabelaları takip ederek park alanına ulaşmaktadır. Araç hedef noktaya düzgün bir şekilde gaz ve direksiyon hamlelerinde bulunarak park alanına ulaşılması test edilmiştir. Gerçek ortamda Şekil-1 'de araç Şerit takip algoritmasını kullanarak sağ ve sol manevralar yapmaktadır ve saha alanında konumunu korumaktadır. Araç sağ tarafa manevra yaparken sol şerit takibini bırakmaktadır ve Açısal değerini koruyarak sağa manevra yapmaktadır. Yapılacak olan ileriki zaman testlerimizde ise aracın kontrollü şekilde hızını artırarak istenilen sürede hedefe varmaktır.

Gerçek araç sürüş testlerinde OpenCV kütüphanesi kullanılarak ZED kamerasında gelen görüntüyü işlemek için ve filtreme yaparak görüntünün düzgün bir şekilde iletilmesini sağlamaktadır. ZED kamerasından gelen görüntü ile şerit takibi ve Tabela okuma işlemleri yapılmaktadır. Okunulan bilgilere göre can bağlantısı yapıp Araç'ın doğru kararlar alıp doğru manevralar ile hedefe ulaşmaktadır.

ZED kamerasından elde edilen nokta bulutları ve derinlik bilgileri kullanılarak aracın belirli bir mesafe içerisinde herhangi bir engel ile karşılaşması durumunda durması sağlanmış ve test edilmiştir. Test videosuna aşağıdaki bağlantı üzerinden ulaşılabilir.

Araç güvenlik test videosu şu şekildedir; https://www.youtube.com/watch?v=cAqyF7e_MBE

11. Referanslar

- [1] PID Blok Diyagramı, https://www.google.com/search?q=pid+formula&client=ubuntu&hs=nIL&sxsrf=AOaemvI3n7PZOBRmvvxHiK1EtPuHuCIrug:1631104898787&source=lnms&tbm=isch&sa=X&ved=2ahUKEwi-_9Sgs-_yAhVtMOwKHYM7CrAQ_AUoAXoECAEQAw&biw=924&bih=912#imgsrc=jSnq0TbSqlT4nM
- [2] PID Blok Diyagramı, <https://tr.wikipedia.org/wiki/PID>
- [3] <https://www.stereolabs.com/docs/sensors/imu/>
- [4] Ackerman, E "Video Friday: Bosch and Cars, ROVs and Whales, and Kuka Arms and Chainsaws". IEEE Spectrum. <https://spectrum.ieee.org/automaton/robotics/industrial-robots/video-friday-875243152>,
- [5] Robot Operating System, <http://www.ros.org/>
- [6] Rosen, R. "Google's Self-Driving Cars: 300,000 Miles Logged, Not a Single Accident Under Computer Control". The Atlantic, <https://www.theatlantic.com/technology/archive/2012/08/googles-self-driving-cars-300-000-miles-logged-not-a-single-accident-under-computer-control/260926/>
Son erişim tarihi: 09 Ağustos 2020
- [7] Tepeköylü, S. 2016. "Mobil Lidar Uygulamaları, Veri İşleme Yazılımları ve Modelleri" Geomatik, 1 (1), 1-7.
- [8] Türkiye İstatistik Kurumu, "Karayolu Trafik Kaza İstatistikleri", "2009-2019", <https://data.tuik.gov.tr/Bulten/Index?p=Karayolu-Trafik-Kaza-Istatistikleri-2019-336284>
- [9] Hui J., "Self-driving car object tracking: Intuition and math behind Kalman Filter" [www.medium.com.https://medium.com/@jonathan_hui/self-driving-object-tracking-intuition-and-the-math-behind-kalman-filter-657d11dd0a90](https://medium.com/@jonathan_hui/self-driving-object-tracking-intuition-and-the-math-behind-kalman-filter-657d11dd0a90).
Son erişim tarihi: 11 Aralık 2020
- [10] Hui J., "Self-driving car: Tracking cars with Extended and Unscented Kalman Filter" [www.medium.com.https://medium.com/@jonathan_hui/self-driving-car-tracking-cars-with-extended-and-unscented-kalman-filter-ced448fd90e2](https://medium.com/@jonathan_hui/self-driving-car-tracking-cars-with-extended-and-unscented-kalman-filter-ced448fd90e2).
Son erişim tarihi: 09 Aralık 2020
- [11] Yurtsever E., Lambert J. 2020. "Survey of Autonomous Driving: Common Practices and Emerging Technologies", IEEE Access.
- [12] Gonzalez D., Perez J., 2015. "Review Motion Planning Techniques for Automated Vehicles", IEEE Transactions on Intelligent Transportation Systems.
- [13] Redmon J., Farhadi A., "YOLO: Real Time Object Detection", <https://pjreddie.com/darknet/yolo/>,
Son erişim tarihi: 03 Aralık 2020

[14] Montemerlo M., Becker J., Bhat S., "Junior: The Stanford Entry in the Urban Challenge", <http://robots.stanford.edu/papers/junior08.pdf>.

[15] Getting Started with ROS and ZED, <https://www.stereolabs.com/docs/ros/>

[16] Jetson AGX Xavier, <https://www.nvidia.com/tr-tr/autonomous-machines/embedded-systems/jetson-agx-xavier/>

[17] Jetson AGX Xavier bilgisayar kurulumu, https://www.clearpathrobotics.com/assets/guides/melodic/jackal/jetson_xavier.html

[18] CAN-BUS Arayüz kurulumu ve aktifleştirme, <https://medium.com/@ramin.nabati/enabling-can-on-nvidia-jetson-xavier-developer-kit-aaaa3c4d99c9>

